LBRO

Law-aware Breach Response Orchestrator

Complete Technical Documentation & Study Guide

A comprehensive reference covering every component, concept, and design decision in the LBRO platform — written for a Computer Science final-year student who wants to understand and explain this system end-to-end.

21 Sections · 100+ Interview Q&As · ASCII Architecture Diagrams
FastAPI · React · PostgreSQL · AWS ECS · ML Classifier · JWT · Terraform

Version 1.0 · 2025 · LBRO Engineering Team

Table of Contents

SECTION 1

Project Overview

What is LBRO?

LBRO stands for **Law-aware Breach Response Orchestrator**. It is a full-stack cybersecurity platform that helps organisations detect, manage, and respond to security incidents. 'Law-aware' means the system understands legal obligations — for example, GDPR requires reporting a data breach within 72 hours. LBRO automates those reminders so nothing is missed.

Real-world analogy: the hospital ER

Think of LBRO as a hospital emergency room for cyber-attacks. When a patient (security incident) arrives, the ER triages by severity, assigns a doctor (analyst), tracks every action, stores evidence, notifies regulators, and records everything for legal purposes. Without such a system, incidents are handled by email and spreadsheets — slow, error-prone, legally risky.

The Problem It Solves

- **Manual triaging is slow.** Without ML classification, analysts manually decide severity.
- **Compliance is forgotten.** GDPR has a 72-hour mandatory window; missing it = fines up to 4% of global revenue.
- **Evidence is fragile.** Security logs can be deleted. LBRO uses tamper-proof S3 WORM storage.
- **No audit trail.** Regulators want to know who did what and when. LBRO records every action.

Tech Stack at a Glance

Layer	Technology	Why
Frontend	React 18 + TypeScript + Vite	Type safety, fast HMR, SPA
State	Zustand + React Query	Auth state + server data cache
Backend	FastAPI (Python 3.11)	Async, auto-docs, fast
ORM	SQLAlchemy 2 async + asyncpg	Non-blocking DB queries
Database	PostgreSQL 15	ACID, JSONB, reliable
ML	scikit-learn + CICIDS2017 pickle	78 features, 15 attack classes
Storage	AWS S3 Object Lock (WORM)	Tamper-proof evidence vault
Queue	AWS SQS	Decoupled async workers
Auth	JWT (python-jose) + bcrypt	Stateless tokens, hashed passwords
Infra	Terraform + ECS Fargate	IaC, serverless containers

Key Numbers

- 9 database tables (incident, user, evidence, audit_log, notification, compliance_obligation, role, permission, api_key)
- 78 CICIDS2017 network features used by the ML classifier

- 15 attack categories the ML model can detect (DDoS, Port Scan, Brute Force, XSS, SQLi, ...)
- 4 RBAC roles: admin, analyst, responder, viewer
- 3 supported regulations: GDPR (72h), HIPAA (60 days), DPDPA (72h)
- Multi-stage Docker builds — non-root user uid 1000 in runtime image
- LocalStack simulates AWS locally: S3, SQS, Secrets Manager

Section Summary

LBRO is a full-stack, law-aware cybersecurity incident response platform combining React frontend, FastAPI backend, PostgreSQL, ML attack classifier, S3 WORM evidence vault, SQS workers, and automated compliance obligation generation.

Interview Questions

- What does 'law-aware' mean in LBRO?
- Why FastAPI over Django or Flask?
- What is SQS's role in the architecture?
- Name the 4 RBAC roles.

Common Mistakes

[!] Confusing the ML classifier with a real-time IDS — it classifies incidents, not live packets.
[!] Thinking LocalStack is production AWS — it is a local simulation only.
[!] Forgetting GDPR's 72-hour clock starts from when you 'become aware', not discovery.

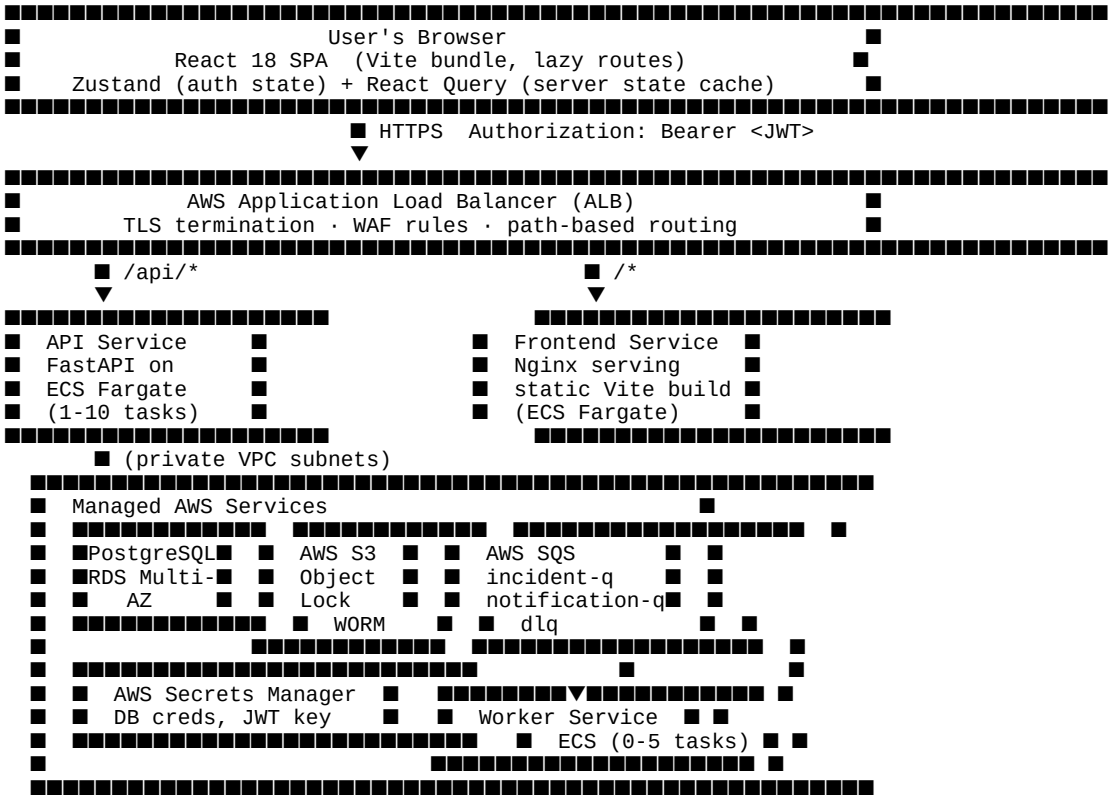
Things to Remember

[v] LBRO = Law-aware Breach Response Orchestrator
[v] 78 features, 15 classes, CICIDS2017 dataset
[v] GDPR: 72h · HIPAA: 60 days · DPDPA: 72h
[v] Frontend: React 18 + Zustand + React Query

SECTION 2

System Architecture

High-Level Architecture Diagram



Layer-by-Layer Explanation

1. React SPA (Browser)

A Single Page Application downloaded once. Vite bundles TypeScript/React into plain JS. All navigation happens in the browser via React Router — no page reloads. Communicates with backend via REST API only.

2. ALB — Application Load Balancer

Internet-facing entry point. Terminates TLS (decrypts HTTPS), applies WAF rules to block malicious requests, and routes: /api/* → API service, /* → frontend static files. Also enforces HTTP request size limits.

3. API Service — FastAPI on ECS Fargate

Handles all business logic: auth, incident CRUD, ML classification triggers, compliance generation, evidence metadata, audit logging. Runs in Docker containers on ECS Fargate (serverless — no EC2 to manage). Auto-scales 1-10 tasks on CPU 70% / Memory 80%.

4. Worker Service — ECS Fargate

Two SQS-polling loops. Incident worker: dequeues new incidents, runs ML classifier, auto-contains criticals. Notification worker: sends notifications with up to 3 retries. Separate from API to avoid blocking API latency.

5. PostgreSQL RDS Multi-AZ

All relational data: incidents, users, evidence metadata, audit logs. Multi-AZ = standby replica in second AZ for automatic failover. Connections use async SQLAlchemy + asyncpg driver.

6. S3 Object Lock WORM

Evidence file storage. COMPLIANCE mode: files cannot be deleted or modified by anyone for the retention period. SHA-256 hash verified at upload.

Local Development Stack (docker-compose)

```
Services in docker-compose.yml:
postgres    → PostgreSQL 15      (port 5432)
localstack  → Fake S3+SQS+SM    (port 4566)
migrate     → alembic upgrade head (one-shot)
api         → FastAPI uvicorn    (port 8000)
worker      → SQS polling loop
frontend    → Vite dev server    (port 5173)
seed        → loads sample data

All on 'lbro-network' bridge. LocalStack simulates
all AWS services with no real AWS account needed.
```

Section Summary

LBRO's architecture has five independent layers: browser SPA, ALB, API service, async worker service, and managed AWS data services. Each scales independently. Locally, docker-compose + LocalStack replicates the full stack.

Interview Questions

- Why separate the worker from the API?
- What is ECS Fargate vs EC2?
- What does Multi-AZ RDS provide?
- What does the ALB route?

Common Mistakes

[!] LocalStack ≠ real AWS — not all features/IAM policies are supported.
[!] ECS auto-scaling is not instant — 1-3 minutes for new tasks to start.
[!] Confusing the ALB with Nginx — they operate at different layers.

Things to Remember

[v] ALB: /api/* → API, /* → frontend
[v] Workers separate from API: protect API latency
[v] S3 COMPLIANCE mode: nobody can delete evidence
[v] ECS auto-scales: CPU 70%, Memory 80%

SECTION 3

Frontend — React, TypeScript, Zustand, React Query

File Structure

```
frontend/src/
■■■ api/client.ts          ← Axios instance + API functions
■■■ hooks/useApi.ts        ← React Query hooks
■■■ pages/                 ← 11 page components (LoginPage, DashboardPage, ...)
■■■ store/authStore.ts     ← Zustand auth state
■■■ routes/AppRouter.tsx   ← Route definitions + lazy loading
■■■ routes/ProtectedRoute  ← Redirects unauthenticated users
■■■ components/ui/         ← Reusable: StatCard, Skeleton, ErrorBoundary
■■■ types/                 ← TypeScript interfaces
■■■ constants/             ← Poll intervals, page sizes
```

Zustand Auth Store

Zustand is a minimal React state management library. The auth store holds the access token, current user object, and login/logout methods. The persist middleware serializes state to sessionStorage (tab-scoped — clears on tab close). A module-level variable _accessTokenMemory mirrors the token for synchronous Axios reads.

```
// store/authStore.ts (simplified)
let _accessTokenMemory: string | null = null

export const useAuthStore = create<AuthState>()(
  persist(
    (set) => ({
      accessToken: null,
      user: null,
      login: async (email, password) => {
        const { access_token } = await authApi.login(email, password)
        _accessTokenMemory = access_token
        set({ accessToken: access_token })
        const user = await authApi.me(access_token)  // explicit token
        set({ user })
      },
      logout: () => { _accessTokenMemory = null; set({ accessToken: null, user: null }) },
    }),
    {
      name: 'lbro-auth',
      storage: createJSONStorage(() => sessionStorage),
      partialize: (s) => ({ accessToken: s.accessToken }),
      onRehydrateStorage: () => (state) => {
        if (state?.accessToken) _accessTokenMemory = state.accessToken
      },
    }
  )
)
export const getAccessToken = () => _accessTokenMemory
```

React Query Hooks

```
// hooks/useApi.ts
export function useIncidents(filters = {}) {
  return useQuery({
    queryKey: ['incidents', 'list', filters],
    queryFn: () => incidentsApi.list(filters),
    refetchInterval: 30_000,   // poll every 30 s
    staleTime: 5_000,
    placeholderData: prev => prev,  // no flash of empty while loading
  })
}

// Infinite scroll for 100k+ incidents
export function useInfiniteIncidents(filters = {}) {
```

```
  return useInfiniteQuery({
    initialPageParam: 1,
    getNextPageParam: (lastPage) =>
      lastPage.page * lastPage.page_size < lastPage.total
        ? lastPage.page + 1 : undefined,
    ...
  })
}
```

Axios Interceptor

```
// api/client.ts
axios.interceptors.request.use((config) => {
  const token = getAccessToken()  // sync read of module variable
  if (token) config.headers['Authorization'] = `Bearer ${token}`
  return config
})
axios.interceptors.response.use(
  res => res,
  err => {
    if (err.response?.status === 401) useAuthStore.getState().logout()
    return Promise.reject(err)
  }
)
```

Route-Based Code Splitting

Each page is loaded lazily with React.lazy() + Suspense. The browser only downloads a page's JS when the user navigates there. A Skeleton fallback shows while loading. An ErrorBoundary wraps each route to catch render errors.

Section Summary

The LBRO frontend is a React 18 SPA using TypeScript, Vite, Zustand, and React Query. Auth state persists in sessionStorage. An Axios interceptor attaches JWT to every request. Pages are code-split for fast initial load.

Interview Questions

- Difference between Zustand and React Query?
- Why is _accessTokenMemory needed?
- What happens on a 401 response?
- Why sessionStorage, not localStorage?

Common Mistakes

[!] Using useAuthStore() inside Axios interceptors — interceptors are synchronous.
[!] Forgetting partialize — persists sensitive data accidentally.
[!] Not handling 401 responses — user stays in broken state after token expiry.

Things to Remember

[v] Zustand = auth/client state · React Query = server state
[v] _accessTokenMemory = sync mirror of token for Axios
[v] sessionStorage = clears on tab close = auto logout
[v] React.lazy + Suspense = route-level code splitting

SECTION 4

Backend — FastAPI, SQLAlchemy, PostgreSQL, Workers

Async Python — Why It Matters

Normal Python code blocks the thread while waiting for I/O (database, network). Async Python uses an event loop — while waiting for the DB, other requests are served. Like a waiter who takes table 1's order, then goes to table 2 while table 1's food cooks.

```
# Sync (blocks thread)            # Async (non-blocking)
def get_incidents(db):             async def get_incidents(db):
    return db.query(...).all()         result = await db.execute(...)
                                       return result.scalars().all()
```

File Structure

```
backend/app/
├── main.py                ← App factory, middleware, lifespan
├── core/
│   ├── config.py          ← pydantic-settings reads env vars
│   ├── database.py        ← async engine + session factory
│   ├── security.py        ← JWT create/decode, bcrypt
│   └── rbac.py            ← roles, permissions, require_permission
├── models/                ← SQLAlchemy ORM (9 tables)
├── routers/               ← FastAPI routers per resource
├── services/              ← Business logic layer
├── middleware/            ← security_headers, rate_limit
├── ml/                    ← classifier, features
├── workers/               ← incident_worker, notification_worker
└── migrations/            ← Alembic versioned migrations
```

App Lifespan

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    await init_db()          # create tables if absent
    await sqs.connect()      # SQS client setup
    classifier.load_model()  # warm up ML model from pickle
    yield                    # app runs here
    await engine.dispose()   # clean up DB pool on shutdown

app = FastAPI(lifespan=lifespan)
```

Dependency Injection

FastAPI injects dependencies declared in endpoint signatures. get_db() creates a DB session, yields it to the endpoint, and closes it automatically — even on exception. require_permission() creates an RBAC guard that runs before the endpoint.

```
async def get_db():
    async with AsyncSessionLocal() as session:
        yield session           # auto-closed after request

@router.get('/incidents')
async def list_incidents(
    filters: IncidentFilters = Depends(),
    db: AsyncSession = Depends(get_db),
    user: User = Depends(require_permission(Permission.READ_INCIDENTS)),
):
    return await IncidentService(db).list(filters)
```

Worker Architecture

```
# incident_worker.py
```

```
async def run_forever():
    while True:
        msgs = await sqs.receive('incident-queue', max=10)
        for msg in msgs:
            incident_id = msg.body['incident_id']
            prediction = classifier.predict(await get_features(incident_id))
            await update_ml_label(incident_id, prediction)
            if prediction.severity == 'critical':
                await auto_contain(incident_id)
            await sqs.delete(msg)
        await asyncio.sleep(1)

# notification_worker.py
async def run_forever():
    while True:
        msgs = await sqs.receive('notification-queue', max=10)
        for msg in msgs:
            success = await send_notification(msg.body)
            if not success and msg.retry_count < 3:
                await sqs.requeue(msg)
            await sqs.delete(msg)
```

Section Summary

FastAPI provides async REST API handling with auto-generated OpenAPI docs. SQLAlchemy 2 with asyncpg provides non-blocking database access. The lifespan context manager handles startup and shutdown. Two SQS-polling workers handle ML classification and notifications asynchronously, isolated from API response times.

Interview Questions

- What is the event loop in async Python?
- What does the lifespan context manager replace?
- How does Depends(get_db) ensure session cleanup?
- What is the producer-consumer pattern in the worker?

Common Mistakes

[!] Mixing sync DB calls in async endpoints blocks the event loop.

[!] Not using Depends(get_db) — session leaks cause connection pool exhaustion.

[!] Forgetting CORS middleware — browser blocks cross-origin requests.

Things to Remember

[v] FastAPI = async-first, auto-OpenAPI docs

[v] Depends(get_db) = creates + auto-closes DB session

[v] Workers poll SQS every 1s; max 3 notification retries

[v] Middleware order: CORS → Security Headers → Rate Limit → Request ID

SECTION 5

Database Design — 9 Tables, Relationships, Migrations

Entity-Relationship Overview

```
users ■■■■■■■■■■■■■■■■ incidents ■■■■■■■■■ evidence
(id, email,           (id, title,         (id, incident_id FK,
 hashed_password,      severity, status,   filename, sha256_hash,
 role,                 ml_label,           file_size, content_type,
 failed_logins,        assigned_to FK,     custody_chain JSONB,
 locked_until)         created_by FK)      uploaded_by FK)
     ■                     ■
     ■                     ■■■ audit_log (who/what/when)
     ■                     ■■■ notification (channel, status, retries)
     ■                     ■■■ compliance_obligation (regulation, deadline)
     ■
api_key (user_id FK, key_hash, last_used)
role_permission (role, permission)  — RBAC mapping table
```

Key Design Decisions

UUIDs as Primary Keys

UUIDs (like 3f2504e0-4f89-11d3-9a0c) instead of integers. Globally unique — generate client-side without DB round-trip. Don't reveal record counts. Safe to use in URLs without enumeration risk.

JSONB for custody_chain

PostgreSQL JSONB stores JSON as binary — indexed and queryable. The custody chain is always read with its evidence record, so a separate table adds unnecessary joins. JSONB stores the array directly on the row.

Account Lockout Columns

failed_logins (integer) and locked_until (timestamp) on the users table. After 5 failures, locked_until = now + 15 minutes. Checked before password verification on every login attempt.

Soft Deletes via status

Incidents are never hard-deleted. They are closed with status='closed'. This preserves the full audit trail for compliance and forensic purposes.

Alembic Migration Example

```
# 001_initial_schema.py
def upgrade():
    op.create_table('users',
        sa.Column('id',              UUID,        primary_key=True, default=uuid4),
        sa.Column('email',           sa.String(255), nullable=False, unique=True),
        sa.Column('hashed_password', sa.String(255), nullable=False),
        sa.Column('role',            sa.String(50),  default='viewer'),
        sa.Column('failed_logins',   sa.Integer,     default=0),
        sa.Column('locked_until',    sa.DateTime,    nullable=True),
        sa.Column('created_at',      sa.DateTime,    default=utcnow),
    )
    op.create_index('ix_users_email', 'users', ['email'], unique=True)

def downgrade():
    op.drop_table('users')
```

SQLAlchemy ORM Model

```
class Incident(Base):
    __tablename__ = 'incidents'
```

```
id          = Column(UUID, primary_key=True, default=uuid4)
title       = Column(String(255), nullable=False)
severity    = Column(Enum('low','medium','high','critical'))
status      = Column(Enum('open','in_progress','contained','resolved','closed'))
ml_label    = Column(String(100), nullable=True)
assigned_to = Column(UUID, ForeignKey('users.id'), nullable=True)
created_at  = Column(DateTime, default=utcnow)
assignee    = relationship('User', foreign_keys=[assigned_to], lazy='selectin')
evidence    = relationship('Evidence', back_populates='incident', lazy='selectin')
```

Section Summary

LBRO uses 9 PostgreSQL tables managed by Alembic migrations. UUIDs are PKs for global uniqueness. custody_chain stored as JSONB. Account lockout in users table. Soft deletes via status field preserve audit trail. lazy='selectin' enables async eager loading.

Interview Questions

- Why UUIDs over auto-increment integers?
- What is JSONB and when would you use it?
- What is the difference between upgrade() and downgrade()?
- Why does SQLAlchemy async need lazy='selectin' instead of lazy='select'?

Common Mistakes

[!] Running raw ALTER TABLE without Alembic — schema and code diverge.

[!] Missing indexes on foreign keys — full table scans on joins.

[!] Using lazy='select' in async SQLAlchemy — causes DetachedInstanceError.

Things to Remember

[v] 9 tables: users, incidents, evidence, audit_log, notification, compliance_obligation, api_key, permission, role_permission

[v] JSONB = binary JSON, indexable in PostgreSQL

[v] Account lockout: 5 failures → locked 15 min

[v] Soft delete: status='closed', never hard delete

SECTION 6

Authentication & Authorization — JWT, RBAC, Sessions

Authentication vs Authorization

Authentication = Who are you? (login with credentials). **Authorization** = What can you do? (permission check per endpoint). LBRO: auth = JWT login. Authorization = RBAC dependency check.

JWT Structure

```
JWT = Header.Payload.Signature
eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiJ1c2VyLWlkIn0 . <HMAC-SHA256>
■■■ algorithm: HS256 ■ ■■■ user_id, role, exp ■■■ ■■■ signed with SECRET_KEY ■

Payload (decoded):
{ "sub": "uuid-of-user", "role": "analyst", "exp": 1735689600, "iat": 1735686000 }

# security.py
def create_access_token(user_id, role, minutes=30):
    return jwt.encode(
        {"sub": str(user_id), "role": role,
         "exp": utcnow() + timedelta(minutes=minutes)},
        SECRET_KEY, algorithm="HS256"
    )

def decode_token(token) -> dict:
    return jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
```

Login Flow

```
POST /api/v1/auth/login
Content-Type: application/x-www-form-urlencoded
Body: username=alice@lbro.com&password=secret

1. OAuth2PasswordRequestForm received
2. Look up user by email in DB
3. Check locked_until — if locked: raise HTTP 423
4. bcrypt.verify(password, user.hashed_password) — constant-time
5. On failure: increment failed_logins; ≥5 → set locked_until = now+15min
6. On success: reset failed_logins = 0
7. Return: { access_token (30min), refresh_token (7days), token_type: "bearer" }

Frontend: stores access_token in sessionStorage via Zustand persist
```

RBAC — 4 Roles, 15+ Permissions

Role	Key Permissions
admin	All permissions — full CRUD on all resources, user management
analyst	Read+update incidents, upload evidence, run ML, view compliance
responder	Read incidents, update status (contain/resolve), upload evidence
viewer	Read-only access to incidents, evidence, notifications

```
# rbac.py — require_permission factory
def require_permission(perm: Permission):
    async def check(user: User = Depends(get_current_user)):
        if perm not in ROLE_PERMISSIONS[user.role]:
            raise HTTPException(403, "Insufficient permissions")
        return user
    return check
# Usage in router:
```

```
@router.delete('/incidents/{id}')
async def delete_incident(
    id: str,
    user = Depends(require_permission(Permission.DELETE_INCIDENTS))
): ...
```

bcrypt — Secure Password Hashing

bcrypt is intentionally slow (work factor). Each stored password includes a unique random salt — same password hashes differently each time, defeating rainbow tables. passlib's verify() uses constant-time comparison to prevent timing attacks (attacker cannot measure how many characters are correct from response time).

Section Summary

JWT tokens carry user ID, role, and expiry. The server never stores tokens — it only validates the HS256 signature. bcrypt with constant-time comparison and account lockout protects against brute force. RBAC checks permissions per endpoint via FastAPI dependencies.

Interview Questions

- What are the three parts of a JWT?
- Why is bcrypt preferred over SHA-256 for passwords?
- What is a timing attack?
- How does require_permission work as a FastAPI dependency?

Common Mistakes

[!] Storing JWT in localStorage — vulnerable to XSS attacks.
[!] Not validating the token signature — allows token forgery.
[!] Using == for password comparison — vulnerable to timing attacks.

Things to Remember

[v] JWT = Header.Payload.Signature — stateless, server never stores
[v] Access: 30 min · Refresh: 7 days
[v] bcrypt: slow by design, includes random salt
[v] 4 roles: admin > analyst > responder > viewer

SECTION 7

Security Layers — Headers, Rate Limiting, CORS, Secrets

Defence in Depth

```
Security Layers (outer → inner):
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 1. AWS WAF          blocks SQLi, XSS, bad user-agents ■
■ 2. ALB              TLS termination, request limits   ■
■ 3. Rate Limiter     100 req/60s per IP, 429+Retry-After■
■ 4. Security Headers X-Frame-Options, CSP, HSTS, etc. ■
■ 5. JWT + RBAC       auth + per-endpoint permission    ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

HTTP Header	Value	Attack Prevented
X-Content-Type-Options	nosniff	MIME sniffing
X-Frame-Options	DENY	Clickjacking
Content-Security-Policy	strict whitelist	XSS, data injection
Strict-Transport-Security	max-age=31536000	SSL stripping
X-Request-ID	per-request UUID	Audit trail correlation

Sliding-Window Rate Limiter

```
# rate_limit.py (simplified)
class SlidingWindowRateLimiter:
    def __init__(self, max_req=100, window=60):
        self.max, self.window = max_req, window
        self.clients: dict[str, list[float]] = {}

    def is_allowed(self, ip: str) -> tuple[bool, int]:
        now = time.time()
        cutoff = now - self.window
        self.clients[ip] = [t for t in self.clients.get(ip,[]) if t > cutoff]
        if len(self.clients[ip]) >= self.max:
            retry = int(self.clients[ip][0] + self.window - now) + 1
            return False, retry   # → 429 with Retry-After: retry
        self.clients[ip].append(now)
        return True, 0
```

CORS Configuration

CORS (Cross-Origin Resource Sharing) allows the React frontend on port 5173 to call the API on port 8000. The browser blocks cross-origin requests unless the server explicitly allows them. In production, only the specific frontend domain is whitelisted (not the wildcard *).

Secrets Management

Sensitive values (DB password, JWT secret, S3 credentials) are stored in AWS Secrets Manager. The app fetches them at startup using boto3. In local development, they are environment variables read by docker-compose; LocalStack's Secrets Manager service simulates the AWS API.

Section Summary

LBRO implements defence-in-depth: WAF → ALB → rate limiting → security headers → JWT+RBAC. The sliding-window rate limiter prevents burst attacks. CORS restricts cross-origin calls to the known frontend domain. Secrets are managed in AWS Secrets Manager, never in code.

Interview Questions

- What is clickjacking and how does X-Frame-Options stop it?
- Fixed-window vs sliding-window rate limiting — what is the boundary burst attack?
- Why can't you use CORS wildcard (*) in production?
- What does HSTS do?

Common Mistakes

[!] Access-Control-Allow-Origin: * in production — any site can call your API.
[!] Not purging old timestamps in rate limiter — unbounded memory growth.
[!] Hardcoding JWT secret in source code — rotatable only with a redeployment.

Things to Remember

[v] 5 layers: WAF → ALB → Rate Limit → Security Headers → JWT+RBAC
[v] Sliding window: timestamps in last 60s per IP, 100 req max
[v] 429 response: Retry-After header tells client when to retry
[v] Secrets Manager: fetch at startup, never hardcode

SECTION 8

Compliance Engine — GDPR, HIPAA, DPDPA

Why Automate Compliance?

Breach response teams are overwhelmed during an incident. Legal deadlines get missed when tracking is manual. LBRO auto-generates compliance obligations the moment an incident is created — giving analysts a visible, timed checklist so no deadline slips.

Regulation	Jurisdiction	Deadline	Max Fine
GDPR	EU / EEA	72 hours to supervisory authority (DPA)	4% global revenue or EUR 20M
HIPAA	USA (healthcare)	60 days to HHS OCR	USD 1.9M/year
DPDPA	India	72 hours to Data Protection Board	INR 250 crore (~USD 30M)

ComplianceService.generate_obligations()

```
REGULATION_RULES = {
    "GDPR":  {"hours": 72,   "authority": "Data Protection Authority"},
    "HIPAA": {"hours": 1440, "authority": "HHS Office for Civil Rights"},
    "DPDPA": {"hours": 72,   "authority": "Data Protection Board India"},
}

async def generate_obligations(incident_id, created_at):
    for reg, rule in REGULATION_RULES.items():
        deadline = created_at + timedelta(hours=rule["hours"])
        await db.add(ComplianceObligation(
            incident_id = incident_id,
            regulation  = reg,
            deadline    = deadline,
            authority   = rule["authority"],
            status      = "pending",
        ))
    await db.commit()
```

Obligation Lifecycle

```
Incident created → 3 obligations auto-generated
     ■
     ■■■ GDPR  obligation: status=pending → deadline in 72h
     ■■■ HIPAA obligation: status=pending → deadline in 60 days
     ■■■ DPDPA obligation: status=pending → deadline in 72h

Analyst views compliance page → countdown timers visible
Analyst reports to authority  → marks obligation "fulfilled"
Deadline passes without action → system marks "missed" (dashboard alert)
```

Section Summary

LBRO auto-generates GDPR (72h), HIPAA (60-day), and DPDPA (72h) compliance obligations on every incident. Each tracks its deadline, reporting authority, and status. Dashboard shows live countdown timers. This is the 'law-aware' core of LBRO.

Interview Questions

- What is GDPR and what does the 72h deadline cover?
- Difference between HIPAA and GDPR reporting requirements?
- What is the DPDPA?

- What happens when a compliance obligation is 'missed'?

Common Mistakes

[!] GDPR clock starts from 'becoming aware', not from initial detection.
[!] Assuming only one regulation applies — GDPR+DPDPA both have 72h but are separate filings.
[!] Not considering UTC for deadline calculation — timezone bugs cause missed deadlines.

Things to Remember

[v] GDPR: 72h · HIPAA: 60 days · DPDPA: 72h
[v] 3 obligations auto-created per incident
[v] Status: pending → fulfilled or missed
[v] GDPR fine: up to 4% global annual revenue

SECTION 9

Machine Learning — CICIDS2017, 78 Features, 15 Classes

The ML Problem

Given a set of 78 statistical features from a network flow, classify the incident into one of 15 attack categories. This is a multi-class classification problem solved with a scikit-learn Random Forest trained on CICIDS2017.

CICIDS2017 Dataset

Canadian Institute for Cybersecurity, 2017. Contains realistic network traffic for 7 days with labelled attack types. Used worldwide as a benchmark for intrusion detection research. LBRO serialises the trained model as a pickle file (cicids2017_rf.pkl) and loads it at startup.

The 15 Attack Classes

- 1. BENIGN
- 2. DDoS
- 3. DoS GoldenEye
- 4. DoS Hulk
- 5. DoS Slowhttptest
- 6. DoS slowloris
- 7. FTP-Patator
- 8. SSH-Patator
- 9. Bot
- 10. PortScan
- 11. Web Attack – Brute Force
- 12. Web Attack – SQL Injection
- 13. Web Attack – XSS
- 14. Infiltration
- 15. Heartbleed

Sample Features (from features.py)

```
FEATURE_NAMES = [
  'Flow Duration',        'Total Fwd Packets',     'Total Bwd Packets',
  'Total Length Fwd Pkts','Total Length Bwd Pkts',
  'Fwd Packet Length Max','Fwd Packet Length Min','Fwd Packet Length Mean',
  'Bwd Packet Length Max','Bwd Packet Length Min','Bwd Packet Length Mean',
  'Flow Bytes/s',         'Flow Packets/s',
  'Flow IAT Mean',        'Flow IAT Std',          'Flow IAT Max',
  'Fwd IAT Total',        'Fwd IAT Mean',          'Fwd IAT Std',
  'Bwd IAT Total',        'Bwd IAT Mean',
  'Fwd PSH Flags',        'Bwd PSH Flags',
  'Fwd URG Flags',        'Bwd URG Flags',
  'Init_Win_bytes_forward','Init_Win_bytes_backward',
  'Average Packet Size',  'Avg Fwd Segment Size',
  ... (78 total)
]
```

AttackClassifier

```
class AttackClassifier:
    def load_model(self):
```

```
        try:
            self.model = pickle.load(open('models/cicids2017_rf.pkl','rb'))
        except FileNotFoundError:
            self.model = None     # heuristic fallback

    def predict(self, features: dict) -> ClassificationResult:
        if not self.model:
            return self._heuristic_fallback(features)
        vector = [features.get(f, 0.0) for f in FEATURE_NAMES]
        label    = self.model.predict([vector])[0]
        severity = SEVERITY_MAP.get(label, 'medium')
        return ClassificationResult(label=label, severity=severity)

    def _heuristic_fallback(self, features):
        if features.get('Flow Packets/s', 0) > 10000:
            return ClassificationResult('DDoS', 'critical')
        if features.get('Fwd PSH Flags', 0) > 50:
            return ClassificationResult('PortScan', 'high')
        return ClassificationResult('BENIGN', 'low')
```

Severity	Attack Classes
Critical	DDoS, Heartbleed, Infiltration
High	DoS GoldenEye, DoS Hulk, Bot
Medium	Brute Force, SQL Injection, XSS, PortScan
Low	DoS slowloris, slowhttptest
Info	BENIGN

Section Summary

The LBRO ML module uses a scikit-learn classifier trained on CICIDS2017, serialized as a pickle file. It accepts 78 numerical network-flow features and classifies into 15 attack categories. Critical predictions trigger auto-containment. A heuristic fallback handles missing model files.

Interview Questions

- What dataset does LBRO use and why?
- What is the heuristic fallback condition for DDoS?
- What happens after a 'critical' prediction?
- Why use pickle for model serialization?

Common Mistakes

[!] Treating the classifier as a real-time IDS — it classifies existing data, not live packets.
[!] Forgetting to load_model() at startup — causes AttributeError on first prediction.
[!] Not providing default 0.0 for missing features — numpy will raise on None.

Things to Remember

[v] CICIDS2017: 78 features, 15 classes, Random Forest
[v] Model loaded at startup from pickle file
[v] DDoS/Heartbleed/Infiltration → critical → auto-contain
[v] Heuristic: Flow Packets/s > 10000 → DDoS

SECTION 10

Evidence Management — WORM, SHA-256, Chain of Custody

What is Digital Evidence?

Evidence in cybersecurity includes: log files, PCAPs, screenshots, malware samples, memory dumps. It must be preserved unmodified to be legally admissible. If tampered, it may be rejected in court and undermine prosecution or regulatory defence.

S3 Object Lock — WORM Storage

AWS S3 Object Lock in COMPLIANCE mode makes files immutable for a set retention period. Nobody — not even AWS root — can delete or modify the file. LBRO sets a 7-year retention period on every evidence upload. This is WORM: Write Once, Read Many.

```
# s3_service.py
await s3.put_object(
    Bucket = EVIDENCE_BUCKET,
    Key    = f"evidence/{incident_id}/{uuid4()}/{filename}",
    Body   = file_bytes,
    ObjectLockMode            = "COMPLIANCE",
    ObjectLockRetainUntilDate = utcnow() + timedelta(days=365*7),
)
```

SHA-256 Integrity Verification

```
# evidence_service.py
sha256 = hashlib.sha256(file_bytes).hexdigest()  # computed pre-upload
# Store sha256_hash in DB evidence row
# On download: recompute hash, compare to stored value
# Mismatch → evidence was tampered with
```

Chain of Custody (JSONB)

```
custody_chain = [
  {
    "action":            "UPLOADED",
    "performed_by_name": "alice@lbro.com",
    "created_at":        "2025-01-15T10:23:00Z",
    "notes":             "Collected from affected server /var/log/auth.log"
  },
  {
    "action":            "DOWNLOADED",
    "performed_by_name": "forensics@firm.com",
    "created_at":        "2025-01-16T14:05:00Z",
    "notes":             "Transferred to external forensics lab"
  },
  {
    "action":            "VERIFIED",
    "performed_by_name": "alice@lbro.com",
    "created_at":        "2025-01-16T14:07:00Z",
    "notes":             "SHA-256 verified post-transfer: MATCH"
  }
]
```

Section Summary

LBRO stores evidence files in S3 COMPLIANCE-mode Object Lock (7-year retention) — legally tamper-proof. SHA-256 hashes are computed at upload and verified on download. Every access is appended to the custody_chain JSONB array, creating an immutable chain of custody for forensic admissibility.

Interview Questions

- Difference between S3 Object Lock GOVERNANCE and COMPLIANCE modes?
- Why SHA-256 and not MD5?
- Why is chain of custody legally important?
- Why store custody_chain as JSONB instead of a separate table?

Common Mistakes

[!] Using GOVERNANCE mode — privileged users can still delete files.
[!] Not verifying hash after download — corrupted file used as evidence undetected.
[!] Storing S3 credentials in source code instead of Secrets Manager.

Things to Remember

[v] COMPLIANCE mode: immutable for retention period, even AWS root cannot delete
[v] SHA-256: 256-bit, collision-resistant — any bit change = different hash
[v] custody_chain: JSONB array, always read with evidence record
[v] 7-year retention = typical legal requirement for forensic evidence

SECTION 11

Notifications — SQS Workers, Retry Logic

What Are Notifications?

When a critical incident occurs, stakeholders must be notified immediately — security team, management, and in some cases regulators. LBRO sends notifications via email and webhook. The sending is decoupled from the API using SQS to avoid slowing down API responses.

Notification Flow

```
1. Analyst triggers notification via POST /api/v1/notifications
2. API creates Notification row (status='pending') in DB
3. API enqueues message to SQS notification-queue
4. API responds immediately (does NOT wait for delivery)
5. Notification worker polls SQS, receives message
6. Worker calls send_notification(channel, recipient, content)
7. On success: update Notification.status = 'sent'
8. On failure: retry_count++; if < 3: requeue to SQS
9. After 3 failures: status='failed', message to DLQ
```

Notification Worker

```
# notification_worker.py (simplified)
async def handle_notification(notification_id):
    notif = await db.get(Notification, notification_id)
    try:
        if notif.channel == 'email':
            await send_email(notif.recipient, notif.subject, notif.body)
        elif notif.channel == 'webhook':
            await send_webhook(notif.webhook_url, notif.payload)
        notif.status = 'sent'
    except Exception as e:
        notif.retry_count += 1
        if notif.retry_count >= 3:
            notif.status = 'failed'
        # else: message stays in SQS for requeue
    await db.commit()
```

Dead Letter Queue (DLQ)

SQS lets you configure a Dead Letter Queue. After a message fails to be processed N times (maxReceiveCount), SQS automatically moves it to the DLQ. Operations teams monitor the DLQ for persistent failures. LBRO sets maxReceiveCount=3 to match the retry logic.

Section Summary

LBRO notifications are decoupled via SQS. The API enqueues a message and returns immediately. The notification worker dequeues, attempts delivery, and retries up to 3 times. Permanent failures land in the DLQ for operations review.

Interview Questions

- Why decouple notifications via SQS instead of sending in the API handler?
- What is a DLQ and when is it used?
- What is the difference between notification status 'sent' and 'failed'?
- What channels does LBRO support for notifications?

Common Mistakes

[!] Sending notifications synchronously in the API — a slow email server blocks the whole response.
[!] Not implementing a DLQ — failed messages loop forever.
[!] Not updating notification.status on success — analytics shows all notifications as pending.

Things to Remember

[v] SQS decouples notification sending from API response
[v] 3 retries max → then DLQ
[v] Channels: email + webhook
[v] maxReceiveCount=3 matches worker retry logic

SECTION 12

Audit Logging — Who Did What and When

Why Audit Logs?

Regulators and legal teams need to know exactly who performed which action on what resource and when. Audit logs are immutable records — you can add to them but never delete entries. They answer: 'Who closed this incident? Who downloaded this evidence? Who changed the severity?'

AuditLog Table Schema

```
class AuditLog(Base):
    __tablename__ = 'audit_log'
    id          = Column(UUID, primary_key=True, default=uuid4)
    incident_id = Column(UUID, ForeignKey('incidents.id'), nullable=True)
    user_id     = Column(UUID, ForeignKey('users.id'), nullable=True)
    action      = Column(String(100))   # e.g. 'incident:update_status'
    resource    = Column(String(100))   # e.g. 'incident'
    resource_id = Column(String(100))   # e.g. the incident UUID
    details     = Column(JSONB)         # before/after state, extra context
    ip_address  = Column(String(45))    # IPv4 or IPv6
    user_agent  = Column(String(255))
    created_at  = Column(DateTime, default=utcnow)
```

What Gets Logged

- User login and logout (with IP address)
- Incident creation, status changes, severity changes
- Evidence upload and download
- User role changes
- API key creation and rotation
- Compliance obligation fulfilment
- Failed login attempts (5th attempt triggers lockout)

Frontend Logging

The frontend also logs important actions to the audit log via the API. The auditAction() helper in logger.ts sends a POST to /api/v1/audit with the action, resource, and context. This captures front-end-initiated actions that might not have corresponding API calls (e.g., viewing sensitive data).

Section Summary

LBRO audit logs capture every security-relevant action: who, what, when, from where. Logs are append-only — entries are never deleted. The details JSONB column stores before/after state for update operations. Both frontend and backend write to the audit log.

Interview Questions

- Why are audit logs never deleted?
- What information does an audit log entry contain?
- Why log from both frontend and backend?
- How does the audit log help with compliance?

Common Mistakes

[!] Deleting old audit logs to save space — violates legal retention requirements.
[!] Not capturing IP address — makes forensic analysis of insider threats impossible.
[!] Logging passwords or tokens in audit details — security/compliance violation.

Things to Remember

[v] Audit log: append-only, never delete
[v] Every entry: user_id, action, resource, resource_id, IP, timestamp
[v] JSONB details: before/after state for updates
[v] Both frontend and backend write audit entries

SECTION 13

Cloud Architecture — AWS ECS, RDS, S3, SQS, ALB

AWS Services Used

Service	What It Does in LBRO
ECS Fargate	Runs API + worker Docker containers (serverless)
ALB	Application Load Balancer — HTTPS entry point, WAF, path routing
RDS PostgreSQL	Managed relational database with Multi-AZ failover
S3 + Object Lock	WORM evidence vault; static frontend hosting
SQS	Message queues: incident-queue, notification-queue, DLQ
Secrets Manager	Stores DB password, JWT secret, S3 credentials
CloudWatch	Logs, metrics, alarms (CPU, memory, 5xx error rate)
VPC + Subnets	Network isolation: public (ALB) + private (ECS, RDS)
WAF	Web Application Firewall — SQLi, XSS, rate rules
IAM	Least-privilege roles for ECS tasks
EventBridge	Scheduled events (e.g., daily compliance checks)
Backup	Automated RDS and S3 backup policies

VPC Network Topology

```
VPC: 10.0.0.0/16
■■■ Public Subnets (10.0.1.0/24, 10.0.2.0/24)
■   ■■■ ALB (internet-facing, EIP)
■   ■■■ NAT Gateways (1 per AZ for outbound traffic)
■
■■■ Private Subnets (10.0.10.0/24, 10.0.11.0/24)
    ■■■ ECS Fargate tasks (API + Worker)
    ■   — no public IP, outbound via NAT
    ■■■ RDS PostgreSQL (Multi-AZ, no internet access)

Security Groups:
- ALB SG:  inbound 443 from 0.0.0.0/0; outbound 8000 to ECS SG
- ECS SG:  inbound 8000 from ALB SG only; outbound 443 + 5432
- RDS SG:  inbound 5432 from ECS SG only
```

ECS Auto-Scaling

```
API Service:
- min_capacity = 1 task, max_capacity = 10 tasks
- Scale OUT when: CPU > 70% OR Memory > 80% for 2 minutes
- Scale IN  when: CPU < 30% AND Memory < 50% for 5 minutes
- cooldown: 300 seconds (prevent thrashing)

Worker Service:
- min_capacity = 1 task, max_capacity = 5 tasks
- Same CPU/Memory thresholds
```

Section Summary

LBRO's cloud infrastructure on AWS uses ECS Fargate for serverless containers, RDS Multi-AZ PostgreSQL for the database, S3 WORM for evidence, SQS for async messaging, ALB+WAF for secure ingress, VPC with public/private subnets for network isolation, and Secrets Manager for credential management.

Interview Questions

- Why is ECS Fargate preferred over EC2 for LBRO?
- What is the purpose of private subnets?
- What triggers ECS auto-scaling?
- Why does RDS need Multi-AZ?

Common Mistakes

[!] Putting RDS in a public subnet — directly reachable from the internet.
[!] Not setting ECS task IAM roles — tasks use the default with too broad permissions.
[!] Not using Secrets Manager — rotating credentials requires redeployment.

Things to Remember

[v] ECS Fargate: no EC2 management, scales per task
[v] Private subnets: ECS + RDS have no public IP
[v] CPU 70% / Memory 80% triggers scale out
[v] RDS Multi-AZ: auto-failover to standby in different AZ

SECTION 14

DevOps — Docker, LocalStack, Terraform, CI/CD

Docker Multi-Stage Build

LBRO uses multi-stage Docker builds. The **builder** stage installs all Python deps (including compilers). The **runtime** stage copies only the installed packages — no compilers, no dev tools. Result: a smaller, more secure image. The runtime runs as uid 1000 (non-root) for security.

```
# docker/Dockerfile.api (simplified)
FROM python:3.11-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

FROM python:3.11-slim AS runtime
WORKDIR /app
RUN adduser --uid 1000 --no-create-home --disabled-password appuser
COPY --from=builder /install /usr/local
COPY backend/app ./app
USER appuser          # run as non-root
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

LocalStack — Local AWS Simulation

LocalStack runs all AWS services locally in a single Docker container. LBRO configures boto3 to point to http://localstack:4566 instead of real AWS. This means developers can run the complete stack locally with no AWS account, no costs, and no risk of touching production resources.

```
# docker-compose.yml — localstack service
localstack:
  image: localstack/localstack:latest
  environment:
    SERVICES: s3,sqs,secretsmanager
    DEFAULT_REGION: us-east-1
  ports: ["4566:4566"]

# backend — boto3 uses localstack endpoint
s3 = boto3.client('s3', endpoint_url='http://localstack:4566')
sqs = boto3.client('sqs', endpoint_url='http://localstack:4566')
```

Terraform Infrastructure as Code

Terraform manages all AWS resources as code. Changes are version-controlled, reviewed, and applied reproducibly. LBRO uses modular Terraform: each concern is a separate module.

```
terraform/modules/
■■■ networking/   → VPC, subnets, NAT gateways, flow logs
■■■ ecs/          → ECS cluster, task defs, services, auto-scaling
■■■ iam/          → Task roles with least-privilege policies
■■■ rds/          → PostgreSQL instance, parameter groups
■■■ s3/           → Evidence bucket with Object Lock config
■■■ sqs/          → Queues, DLQ, redrive policies
■■■ monitoring/   → CloudWatch dashboards, alarms, log groups
■■■ waf/          → WAF ACL rules
■■■ eventbridge/  → Scheduled rules
■■■ backup/       → Backup plans and vaults
■■■ secrets/      → Secrets Manager secrets
```

Section Summary

LBRO uses multi-stage Docker builds for minimal runtime images (uid 1000 non-root). LocalStack simulates all AWS services locally for zero-cost development. Terraform manages all AWS infrastructure as code across 11

modules. docker-compose runs the complete local stack including all services and seed data.

Interview Questions

- What is the benefit of multi-stage Docker builds?
- What does LocalStack simulate and what are its limitations?
- What is Infrastructure as Code and why use Terraform?
- What is the difference between Terraform plan and apply?

Common Mistakes

[!] Running as root in production Docker containers — compromised app = root system access.

[!] Thinking LocalStack supports all AWS features — some advanced IAM and STS features are not supported.

[!] Not using .dockerignore — accidentally copies node_modules or .env into the image.

Things to Remember

[v] Multi-stage: builder (compilers) + runtime (clean image, uid 1000)

[v] LocalStack: endpoint_url='http://localstack:4566'

[v] 11 Terraform modules for clean separation of concerns

[v] terraform plan (preview) → terraform apply (deploy)

SECTION 15

Testing Strategy — pytest, Fixtures, Coverage

Test Architecture

LBRO uses pytest + pytest-asyncio for async tests. The test database is SQLite in-memory (not PostgreSQL) — faster and self-contained. The FastAPI app is tested with httpx's ASGITransport — real HTTP request/response cycle without starting a real server.

```
# conftest.py
@pytest.fixture(scope='session')
def event_loop():
    return asyncio.new_event_loop()

@pytest.fixture(scope='session')
async def db_engine():
    engine = create_async_engine('sqlite+aiosqlite:///:memory:')
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)
    yield engine
    await engine.dispose()

@pytest.fixture
async def client(db_engine):
    # Override get_db to use test DB
    app.dependency_overrides[get_db] = override_get_db
    async with AsyncClient(
        transport=ASGITransport(app=app), base_url='http://test'
    ) as c:
        yield c
```

User Fixtures

```
@pytest.fixture
async def admin_user(db):
    user = User(email='admin@test.com', role='admin',
                hashed_password=bcrypt.hash('password123'))
    db.add(user); await db.commit()
    return user

@pytest.fixture
async def admin_token(admin_user):
    return create_access_token(admin_user.id, 'admin')

@pytest.fixture  # also analyst_user, viewer_user for RBAC testing
```

Test Examples

```
# test_auth.py
async def test_login_success(client):
    resp = await client.post('/api/v1/auth/login',
        data={'username': 'admin@test.com', 'password': 'password123'})
    assert resp.status_code == 200
    assert 'access_token' in resp.json()

async def test_login_wrong_password(client, admin_user):
    resp = await client.post('/api/v1/auth/login',
        data={'username': 'admin@test.com', 'password': 'wrongpass'})
    assert resp.status_code == 401

async def test_protected_endpoint_no_token(client):
    resp = await client.get('/api/v1/incidents')
    assert resp.status_code == 401

async def test_viewer_cannot_delete(client, viewer_token):
    resp = await client.delete('/api/v1/incidents/some-id',
        headers={'Authorization': f'Bearer {viewer_token}'})
    assert resp.status_code == 403
```

Section Summary

LBRO uses pytest + pytest-asyncio with an SQLite in-memory test database. FastAPI is tested via httpx ASGITransport — real request/response without a running server. Fixtures create users of each role and their tokens. RBAC tests verify that role boundaries are enforced on every sensitive endpoint.

Interview Questions

- Why use SQLite in-memory for tests instead of PostgreSQL?
- What is ASGITransport and why is it used?
- What is dependency_overrides used for in tests?
- How do you test RBAC — that a viewer cannot delete?

Common Mistakes

[!] Not resetting the database between tests — one test's data pollutes another's assertions.
[!] Using a real PostgreSQL for tests — slow, requires a running DB, hard to parallelize.
[!] Not testing role boundaries — viewer might silently be able to delete in production.

Things to Remember

[v] SQLite in-memory: fast, isolated, no server needed
[v] ASGITransport: test FastAPI without starting a server
[v] dependency_overrides: swap real get_db for test DB
[v] Test every role boundary (admin/analyst/responder/viewer)

SECTION 16

API Documentation — Endpoints, Schemas, Error Codes

API Design Principles

- **RESTful** — resources (incidents, evidence) as nouns; HTTP verbs as actions
- **Versioned** — all endpoints under /api/v1/ to allow breaking changes in future
- **Consistent responses** — all list endpoints return PagedResponse; errors follow RFC 7807
- **Auto-documented** — FastAPI generates OpenAPI 3.0 schema from Python type hints

Core Endpoints

Method	Path	Permission	Description
POST	/auth/login	public	Login, receive JWT
POST	/auth/refresh	public	Exchange refresh for new access token
GET	/auth/me	any authenticated	Current user profile
GET	/incidents	incidents:read	Paginated list with filters
POST	/incidents	incidents:create	Create new incident
GET	/incidents/{id}	incidents:read	Single incident detail
PATCH	/incidents/{id}	incidents:update	Update title/severity/status
DELETE	/incidents/{id}	incidents:delete	Soft-delete (status=closed)
GET	/incidents/{id}/evidence	incidents:read	List evidence for incident
POST	/incidents/{id}/evidence	evidence:upload	Upload evidence file
GET	/incidents/{id}/compliance	compliance:read	Compliance obligations
GET	/incidents/{id}/audit	audit:view	Audit log entries
GET	/health	public	Health check
GET	/health/ready	public	Readiness check (DB+SQS)

Paged Response Schema

```
{
  "total":     150,     // total matching records
  "page":      1,       // current page (1-indexed)
  "page_size": 20,      // items per page
  "items":     [...]    // array of resource objects
}
// Note: NO has_next or has_prev fields — derive from page * page_size < total
```

Error Response Format (RFC 7807)

```
HTTP 422 Unprocessable Entity:
{ "detail": [{"loc":["body","email"],"msg":"field required","type":"missing"}] }

HTTP 401 Unauthorized:
{ "detail": "Could not validate credentials" }
HTTP 403 Forbidden:
```

```
{ "detail": "Insufficient permissions" }

HTTP 429 Too Many Requests:
Retry-After: 42
{ "detail": "Rate limit exceeded. Try again in 42 seconds." }
```

Section Summary

The LBRO REST API follows RESTful conventions under /api/v1/. All list endpoints return PagedResponse (total, page, page_size, items). FastAPI auto-generates OpenAPI docs at /docs. Error responses follow RFC 7807 with consistent HTTP status codes.

Interview Questions

- What is the difference between 401 and 403?
- Why version the API with /v1/?
- What does PagedResponse contain and what does it NOT have?
- How does FastAPI generate the OpenAPI spec automatically?

Common Mistakes

[!] Returning 200 for errors — breaks clients that check status codes.

[!] Using has_next/has_prev — PagedResponse has no such fields; derive from page*page_size less than total.

[!] Not versioning the API — breaking changes in v2 would break all v1 clients.

Things to Remember

[v] All endpoints: /api/v1/...

[v] PagedResponse: total, page, page_size, items — no has_next/has_prev

[v] 401 = not authenticated · 403 = not authorized

[v] FastAPI auto-docs: /docs (Swagger) and /redoc

SECTION 17

Complete Request Lifecycle — Login to Incident Resolution

Trace 1: Login

```
Browser → POST /api/v1/auth/login (form-encoded: username, password)
     ■
     ▼ ALB
[TLS termination → WAF rules → forward to ECS task]
     ■
     ▼ Rate Limiter Middleware
[check sliding window: IP not over 100 req/min → allow]
     ■
     ▼ Security Headers Middleware
[add X-Frame-Options, CSP, HSTS, X-Request-ID headers]
     ■
     ▼ FastAPI Router: POST /auth/login
[OAuth2PasswordRequestForm parsed]
     ■
     ▼ auth_service.login(email, password)
[DB: SELECT user WHERE email=?]
[check locked_until — not locked]
[bcrypt.verify(password, hashed_password) — match]
[reset failed_logins = 0]
[create_access_token(user.id, user.role) → JWT string]
[create_refresh_token(user.id) → refresh JWT]
     ■
     ▼ Response
{ "access_token": "eyJ...", "refresh_token": "eyJ...", "token_type": "bearer" }
[Response headers: X-Frame-Options: DENY, X-Request-ID: uuid4, ...]
```

Trace 2: Create Incident (authenticated)

```
Browser → POST /api/v1/incidents
Headers: Authorization: Bearer eyJ...
Body: { "title": "SSH Brute Force", "severity": "high", "description": "..." }
     ■
     ▼ Axios Interceptor (frontend)
[getAccessToken() → reads _accessTokenMemory module variable]
[config.headers['Authorization'] = 'Bearer eyJ...']
     ■
     ▼ JWT Authentication (FastAPI dependency)
[decode_token(token) → payload: {sub: user_id, role: 'analyst', exp: ...}]
[exp check: not expired]
[DB: SELECT user WHERE id=user_id → user object]
     ■
     ▼ RBAC Check: require_permission(Permission.CREATE_INCIDENTS)
[ROLE_PERMISSIONS['analyst'] contains CREATE_INCIDENTS → allow]
     ■
     ▼ IncidentService.create(payload, created_by=user.id)
[INSERT INTO incidents (...) VALUES (...)]
[INSERT INTO audit_log (...) → action='incident:create']
[ComplianceService.generate_obligations(incident.id, incident.created_at)]
[→ INSERT 3 rows: GDPR 72h, HIPAA 60d, DPDPA 72h]
[sqs.enqueue_incident(incident.id) → message to SQS incident-queue]
     ■
     ▼ Response: 201 Created
{ "id": "uuid", "title": "SSH Brute Force", "severity": "high", "status": "open", ... }
     ■
     ▼ Worker Service (asynchronous, separate ECS task)
[poll SQS → receive incident_id]
[classifier.predict(features) → label='SSH-Patator', severity='high']
[UPDATE incidents SET ml_label='SSH-Patator' WHERE id=?]
[severity != 'critical' → no auto-contain]
```

Section Summary

A single incident creation triggers: JWT auth → RBAC check → DB insert → audit log → 3 compliance obligations → SQS enqueue → async ML classification in the worker. Each layer is decoupled and independently traceable via the X-Request-ID header.

Interview Questions

- What happens between the browser sending the login request and receiving the JWT?
- At which point is the JWT validated in the incident creation request?
- What is the sequence of DB writes when an incident is created?
- Why is ML classification done asynchronously rather than in the API handler?

Common Mistakes

[!] Doing ML classification synchronously in the API — a 500ms model inference blocks the response.
[!] Not creating compliance obligations at incident creation time — deadline clock starts immediately.
[!] Not logging the create action in audit_log — no record of who created what.

Things to Remember

[v] Login: form-encoded → bcrypt verify → JWT issued
[v] Request: Axios injects Bearer → FastAPI decodes JWT → RBAC checks permission
[v] Incident create: DB insert + audit log + 3 compliance obligations + SQS enqueue
[v] ML classification: async via SQS worker, 1-10s after creation

SECTION 18

Common Bugs & How to Fix Them

TypeScript: TS2339 — Property does not exist

WHAT: Accessing a field that doesn't exist on a type. EXAMPLE: ev.has_next where PagedResponse has no such field. FIX: Derive from page * page_size < total. Or update the TypeScript interface to match the real API response.

TypeScript: TS2322 — Type mismatch on props

WHAT: Passing wrong type to a component prop. EXAMPLE: Passing className to Skeleton which only accepts style, height, width. FIX: Check the component's Props interface; use inline styles.

FastAPI: 422 Unprocessable Entity on login

WHAT: Login endpoint uses OAuth2PasswordRequestForm which expects application/x-www-form-urlencoded, but frontend sends JSON. FIX: Set Content-Type: application/x-www-form-urlencoded and encode body as URLSearchParams.

SQLAlchemy: DetachedInstanceError

WHAT: Accessing a relationship attribute after the session is closed. EXAMPLE: accessing incident.assignee outside the request context. FIX: Use lazy='selectin' to eagerly load relationships within the session.

Windows: Trailing null bytes in files

WHAT: Edit/Write tools on Windows-mounted paths append null bytes (\x00). File appears truncated. FIX: python3 -c "raw=open(f,'rb').read().rstrip(b'\x00'); open(f,'wb').write(raw)"

Zustand: Token not available in Axios interceptor

WHAT: useAuthStore() cannot be called in Axios interceptors (they are synchronous). FIX: Mirror the token in a module-level variable (_accessTokenMemory) and read that from the interceptor.

SQS: Messages reprocessed after failure

WHAT: Worker crashes mid-processing; SQS visibility timeout expires; message redelivered. FIX: Make handlers idempotent — check if the incident is already classified before running ML again.

Docker: Container runs as root

WHAT: Default Docker containers run as root — a container escape = root access. FIX: Add RUN adduser --uid 1000 appuser and USER appuser to Dockerfile.

CORS: Blocked by browser

WHAT: API is missing CORS headers for the frontend origin. FIX: Add CORSMiddleware to FastAPI with allow_origins=[FRONTEND_URL].

Rate limiter: Memory leak

WHAT: Old timestamps never purged from the sliding window dict. FIX: Always filter self.clients[ip] to only keep timestamps within the window.

Section Summary

The most common bugs in LBRO relate to TypeScript type mismatches (stale field names), Windows filesystem null bytes from Edit tools, JWT interceptor synchronicity, and SQLAlchemy async session lifecycle. Understanding each bug's root cause prevents recurrence.

Interview Questions

- Why does PagedResponse not have has_next?
- What causes DetachedInstanceError in async SQLAlchemy?
- How do you make an SQS message handler idempotent?
- Why does the Axios interceptor need a module variable for the token?

Common Mistakes

[!] Assuming null bytes in files mean corruption — it's a Windows filesystem tool artifact.

[!] Re-running ML classification on already-classified incidents — wastes compute, may overwrite correct labels.

[!] Catching all exceptions in the worker and silently dropping them — messages lost, no DLQ.

Things to Remember

[v] Windows null bytes: python rstrip(b'\x00') to fix

[v] 422 on login: use form-encoded not JSON

[v] DetachedInstanceError: use lazy='selectin'

[v] Axios interceptor: synchronous — use module variable

SECTION 19

Interview Preparation — 100 Questions & Answers

General & Project

Q: What is LBRO?

A: Law-aware Breach Response Orchestrator — a full-stack cybersecurity incident response platform that combines ML-based attack classification, WORM evidence storage, automated compliance obligation generation (GDPR/HIPAA/DPDPA), and a React dashboard.

Q: Why is it called 'law-aware'?

A: Because it understands legal obligations. When an incident is created, it auto-generates compliance deadlines: GDPR (72h), HIPAA (60 days), DPDPA (72h) — reminding teams before they miss reporting windows.

Q: What was the biggest technical challenge?

A: Async-first design throughout the entire stack — async SQLAlchemy, async FastAPI endpoints, async SQS workers — while keeping the code clean and avoiding common pitfalls like DetachedInstanceError.

Q: How does LBRO differ from a traditional SIEM?

A: A SIEM collects and alerts on logs in real-time. LBRO is an incident response orchestration platform — it manages incidents after detection, with ML triage, evidence management, compliance tracking, and audit logging.

Q: What would you change if you had more time?

A: Real-time WebSocket updates instead of polling, a more sophisticated ML model (e.g., BERT for log analysis), and multi-tenancy for multiple organisations.

FastAPI & Python Backend

Q: Why FastAPI over Django?

A: FastAPI is async-first with native support for asyncio and type hints. Django was designed for sync code and its async support is still maturing. FastAPI also auto-generates OpenAPI documentation and is significantly faster.

Q: What is an async context manager?

A: A Python construct used with 'async with'. It runs async code at entry and exit. Used in FastAPI's lifespan (startup/shutdown) and SQLAlchemy's session factory (yield session, then close).

Q: What is dependency injection in FastAPI?

A: Declaring a Depends() parameter in an endpoint signature. FastAPI resolves, injects, and cleans up the dependency. Example: Depends(get_db) creates a DB session, injects it, and closes it after the response.

Q: How does FastAPI handle request validation?

A: Via Pydantic models. If a request body doesn't match the model's type annotations, FastAPI automatically returns HTTP 422 with a detailed list of validation errors.

Q: What is pydantic-settings?

A: A library for reading config from environment variables with type validation. LBRO's Settings class has DATABASE_URL, JWT_SECRET, etc. — all read from env vars at startup.

Q: What is asyncpg?

A: A pure-Python async PostgreSQL driver. Significantly faster than psycopg2 because it uses native PostgreSQL protocol without ctypes. Required for SQLAlchemy async.

Q: How does SQLAlchemy async differ from sync?

A: All DB calls must be awaited. Sessions are AsyncSession. Relationships must use lazy='selectin' or be explicitly loaded within the session context to avoid DetachedInstanceError.

Q: What is Alembic?

A: A database migration tool for SQLAlchemy. Each migration has upgrade() and downgrade() functions. Run with 'alembic upgrade head' to apply all pending migrations.

Q: What is lazy='selectin' in SQLAlchemy?

A: An async-safe eager loading strategy. When you load an incident, SQLAlchemy immediately runs a second SELECT to load the related objects (e.g., evidence) within the same session, before yielding the result.

Q: What is the lifespan context manager?

A: A FastAPI feature that runs startup code before the app accepts requests and shutdown code when the server stops. Replaces the deprecated @app.on_event() decorators.

Authentication & Security

Q: Explain JWT end-to-end.

A: User logs in with username+password. Server verifies bcrypt hash. Server creates JWT with user ID, role, expiry — signed with a secret key using HS256. Client stores token. On each request, client sends 'Authorization: Bearer token'. Server decodes and verifies signature and expiry.

Q: What is the difference between authentication and authorization?

A: Authentication: verifying identity (who are you?). Authorization: verifying permissions (what can you do?). JWT handles authentication; RBAC handles authorization.

Q: Why is bcrypt better than SHA-256 for passwords?

A: SHA-256 is fast — GPUs can try billions/sec. bcrypt is deliberately slow (work factor) and includes a random salt. Same password hashes differently each time, defeating precomputed (rainbow table) attacks.

Q: What is a timing attack?

A: An attacker measures response time to infer information. If password comparison exits early on the first wrong character, wrong passwords are rejected faster than partially-correct ones, leaking information. bcrypt's verify() uses constant-time comparison.

Q: What is RBAC?

A: Role-Based Access Control. Users have a role (admin/analyst/responder/viewer). Roles map to permission sets. Endpoints check: does this user's role include the required permission? If not, 403 Forbidden.

Q: Why use sessionStorage instead of localStorage for JWT?

A: sessionStorage is tab-scoped — cleared when the browser tab is closed. localStorage persists forever, which is a security risk for auth tokens. sessionStorage provides implicit logout on browser close.

Q: What is a refresh token?

A: A long-lived token (7 days) that can be exchanged for a new short-lived access token (30 min). The access token is short-lived to limit exposure if stolen; the refresh token lets users stay logged in without re-entering their password.

Q: What does X-Frame-Options: DENY do?

A: Prevents the page from being embedded in an on another domain. This stops clickjacking attacks where an attacker overlays invisible buttons on your site.

Q: What is CSP?

A: Content-Security-Policy. An HTTP header that whitelists allowed sources for scripts, styles, images, etc. Prevents XSS by blocking execution of injected scripts from unknown sources.

Q: What is HSTS?

A: HTTP Strict Transport Security. Tells browsers to only ever connect to this domain over HTTPS for the specified duration (max-age). Prevents SSL stripping attacks where an attacker downgrades HTTPS to HTTP.

Database & PostgreSQL

Q: Why UUIDs over auto-increment IDs?

A: UUIDs are globally unique across tables and systems. They can be generated client-side without a DB round-trip. They don't leak record counts via sequential IDs. No collision when merging records from multiple systems.

Q: What is JSONB in PostgreSQL?

A: Binary JSON — stored in parsed binary form rather than text. Indexed and queryable with operators like @> and ->. Faster than JSON type for reads. Used for custody_chain to avoid a separate junction table.

Q: What is ACID?

A: Atomicity (all-or-nothing), Consistency (valid state after transaction), Isolation (concurrent transactions don't interfere), Durability (committed data survives crashes). PostgreSQL is ACID-compliant.

Q: What is Multi-AZ RDS?

A: AWS keeps a synchronous standby replica in a second Availability Zone. On primary failure, AWS automatically fails over to the standby with typically <60s downtime.

Q: What is an Alembic downgrade?

A: The downgrade() function in a migration script reverts the changes made by upgrade(). Allows rolling back to a previous schema version if a deployment fails.

Q: What index is on the users table?

A: ix_users_email — a unique index on the email column. Makes login lookups (SELECT WHERE email=?) O(log n) instead of O(n), and enforces uniqueness at the DB level.

Machine Learning

Q: What is CICIDS2017?

A: Canadian Institute for Cybersecurity Intrusion Detection System 2017 — a benchmark network intrusion dataset with 78 features and 15 labelled attack classes used worldwide for ML-based IDS research.

Q: What algorithm is used?

A: Random Forest trained on CICIDS2017 features, serialised as a scikit-learn pickle file. Loaded at startup; predict() takes a feature vector and returns a label + severity.

Q: What are the 78 features?

A: Statistical properties of network flows: packet counts, byte counts, inter-arrival times (IAT), TCP flag counts, window sizes, segment sizes. Not raw packets — derived statistics from flow aggregation.

Q: What is the heuristic fallback?

A: When the model file is missing, the classifier falls back to rules: Flow Packets/s > 10000 → DDoS (critical); Fwd PSH Flags > 50 → PortScan (high); else BENIGN (low).

Q: What triggers auto-containment?

A: When the ML classifier returns severity='critical' (for DDoS, Heartbleed, Infiltration), the incident worker calls auto_contain(), which isolates affected systems and updates incident status to 'contained'.

Cloud & Infrastructure

Q: What is ECS Fargate?

A: AWS Elastic Container Service in Fargate mode — runs Docker containers without managing EC2 instances. You define task definitions (CPU/memory/image) and Fargate handles the infrastructure.

Q: What triggers ECS auto-scaling?

A: CPU utilisation > 70% or Memory utilisation > 80% for 2 consecutive minutes triggers scale-out. Scale-in when both drop below 30%/50% for 5 minutes.

Q: What is S3 Object Lock COMPLIANCE mode?

A: Files stored in COMPLIANCE mode cannot be deleted or modified by anyone — including the AWS root account — until the retention period expires. Used for WORM forensic evidence storage.

Q: What is the purpose of the DLQ?

A: Dead Letter Queue — SQS automatically moves messages to the DLQ after they fail to be processed maxReceiveCount (3) times. Operations teams monitor it for persistent failures that need manual intervention.

Q: What is Terraform?

A: Infrastructure as Code tool. Defines AWS resources in HCL files, tracks state, and applies changes idempotently. 'plan' shows what will change; 'apply' makes changes.

Q: What does LocalStack do?

A: Runs AWS service emulators (S3, SQS, Secrets Manager) locally. Allows full local development with no AWS account or costs. boto3 is configured with endpoint_url='http://localstack:4566'.

Q: What is a NAT Gateway?

A: Allows ECS tasks in private subnets to make outbound internet connections (e.g., to download packages, call external APIs) without having a public IP address or being directly reachable from the internet.

Q: What is WAF?

A: Web Application Firewall — inspects HTTP requests at the ALB and blocks common attacks: SQL injection patterns, XSS strings, malformed requests, known bad IPs, rate limit violations.

Frontend & React

Q: What is React Query?

A: A server state management library. Handles fetching, caching, background refresh, loading/error states, and polling. Eliminates manual useEffect+useState+loading patterns for server data.

Q: What is code splitting in React?

A: Using React.lazy() to load page components only when navigated to. Each page becomes a separate JS chunk downloaded on demand. Reduces initial bundle size and improves first-load performance.

Q: What is a React Query query key?

A: A unique identifier for a cached query — usually an array like ['incidents', 'list', {status: 'open'}]. React Query uses this to cache, invalidate, and refetch data. Changes in the key trigger a new fetch.

Q: What is useInfiniteQuery?

A: A React Query hook for infinite scroll / pagination. getNextPageParam calculates the next page number. Data accumulates across pages in an InfiniteData structure. Used for 100k+ incident lists.

Q: Why does LBRO use Zustand over Redux?

A: Zustand is minimal — no boilerplate, no action creators, no reducers. For a small auth store (token + user + login/logout), Zustand is far simpler. Redux is better for very complex state trees with many interactions.

Compliance & Evidence

Q: What is the GDPR 72-hour rule?

A: Under GDPR Article 33, a personal data breach must be reported to the supervisory authority within 72 hours of becoming aware. LBRO auto-generates this obligation and shows a countdown timer.

Q: What is HIPAA?

A: Health Insurance Portability and Accountability Act — US law protecting healthcare data. Breaches must be reported to HHS OCR within 60 days. LBRO generates this obligation for all incidents.

Q: What is DPDPA?

A: India's Digital Personal Data Protection Act 2023. Personal data breaches must be reported to the Data Protection Board within 72 hours. Similar to GDPR but for Indian entities.

Q: What is WORM storage?

A: Write Once Read Many — once written, data cannot be modified or deleted. AWS S3 Object Lock in COMPLIANCE mode implements WORM for LBRO's evidence vault.

Q: What is a chain of custody?

A: A documented log of every person who has accessed or handled evidence, with timestamps and reasons. Required for forensic evidence to be legally admissible. LBRO stores it as a JSONB array in the evidence table.

Q: Why store SHA-256 hashes?

A: SHA-256 is a cryptographic hash that uniquely fingerprints a file. Store the hash at upload time; recompute on download. A mismatch proves the file was modified — inadmissible as evidence.

Testing

Q: Why use SQLite for tests instead of PostgreSQL?

A: SQLite in-memory is instant to create, requires no running server, and is destroyed after the test session. Tests run faster and in complete isolation. Schemas are created via Alembic/SQLAlchemy from the same models.

Q: What is ASGITransport?

A: httpx's adapter that lets you send HTTP requests directly to a FastAPI app without starting a real server. Tests use full request/response cycles (including middleware) at in-process speed.

Q: What is a pytest fixture?

A: A reusable setup function declared with @pytest.fixture. Fixtures can yield (setup + teardown), have scope (function/session), and are injected by name into test parameters. LBRO uses fixtures for DB, users, tokens.

Q: How do you test RBAC?

A: Create fixtures for each role (admin_user, analyst_user, viewer_user) and their tokens. Test that viewer gets 403 on DELETE endpoints, responder gets 403 on admin endpoints, etc.

SECTION 20

Project Walkthrough — Explaining It to an Interviewer

The 2-Minute Elevator Pitch

LBRO is a full-stack cybersecurity incident response platform I built to solve three real problems: slow manual incident triage, missed legal compliance deadlines, and fragile evidence storage. It uses a React 18 frontend with Zustand and React Query, a FastAPI async backend, PostgreSQL for data, scikit-learn for ML-based attack classification using the CICIDS2017 dataset, AWS S3 Object Lock for tamper-proof evidence, and SQS for async notification and classification workers. The whole infrastructure is defined in Terraform and runs on ECS Fargate. Locally, docker-compose with LocalStack replaces all AWS services.

Explaining the Tech Choices

FastAPI over Django

I chose FastAPI because the backend is I/O-bound — it spends most time waiting for the database and AWS services. FastAPI's async-first model lets one process handle hundreds of concurrent requests without blocking. FastAPI also generates interactive API docs automatically from my type hints, which saved significant documentation time.

Zustand over Redux

For the frontend auth state (just a token, user object, and two methods), Redux would be massive overkill with action creators and reducers. Zustand gives me a typed store in about 40 lines with built-in persistence middleware.

React Query over plain useEffect

Managing server state with useEffect means writing loading/error/stale flags, polling intervals, and cache invalidation manually on every page. React Query handles all of this with a single useQuery call and gives me stale-while-revalidate, background polling, and query key-based cache invalidation for free.

JWT over sessions

Sessions require server-side storage, which doesn't scale horizontally across ECS tasks without a shared Redis. JWT is stateless — any ECS task can validate any token using the shared secret from Secrets Manager. No sticky sessions needed.

SQS over direct HTTP

Calling notification services synchronously in the API would block the response if the email server is slow. SQS decouples the API from the notification worker. If the worker is down, messages are retained in SQS. This also enables retry logic and DLQ.

Handling Difficult Interview Questions

'What would you do differently?'

I would replace polling (refetchInterval) with WebSockets for real-time incident updates. I would also add proper OAuth2 social login instead of username/password only, and containerize the frontend as a multi-stage Nginx build with Brotli compression.

'How does it scale to 1 million incidents?'

The database needs sharding or partitioning by created_at (PostgreSQL table partitioning). The ML classifier should move to a dedicated microservice so model upgrades don't require API redeployment. Elasticsearch for full-text search on incident descriptions.

'Is it production-ready?'

Mostly — it has proper auth, RBAC, WORM evidence, audit logging, compliance obligations, and IaC. It needs: E2E tests, blue-green deployment, enhanced monitoring (distributed tracing with OpenTelemetry), and security penetration testing.

Section Summary

To explain LBRO: lead with the business problem (slow triage, missed compliance deadlines, fragile evidence), then explain the solution (ML triage, automated obligations, WORM storage), then justify each tech choice with a clear reason. Practice the 2-minute pitch until it's fluent.

Interview Questions

- Why did you choose FastAPI over Flask?
- How would you make LBRO multi-tenant?
- What is the hardest bug you fixed in this project?
- How would you add real-time updates?

Common Mistakes

[!] Starting with implementation details instead of the problem — interviewers want context first.
[!] Not knowing why you chose a technology — every choice must have a reason.
[!] Saying 'it scales infinitely' — everything has limits; show you know them.

Things to Remember

[v] Start with: problem → solution → tech choices
[v] Know your numbers: 78 features, 15 classes, 9 tables, 4 roles
[v] Justify every tech choice with a reason
[v] Have an honest answer for 'what would you change'

SECTION 21

Future Improvements & Engineering Roadmap

Real-time WebSocket Updates

Replace polling (React Query refetchInterval) with WebSocket connections for instant incident updates. Use FastAPI's WebSocket support or AWS API Gateway WebSockets for production.

Advanced ML Model

Replace the CICIDS2017 Random Forest with a transformer-based model trained on recent threat intelligence. Consider BERT for log analysis or a custom neural network for temporal sequence data.

Multi-tenancy

Support multiple organisations in one LBRO instance. Add an organisation_id FK to all tenant-scoped tables. Use row-level security (RLS) in PostgreSQL to enforce tenant isolation at the DB layer.

SSO / OAuth2

Add SAML/OIDC integration for enterprise SSO (Okta, Azure AD). Remove username/password dependency for large organisations with existing IdP infrastructure.

E2E Tests with Playwright

Add end-to-end browser tests covering the critical paths: login, create incident, upload evidence, verify compliance obligations. Run on CI on every pull request.

Blue-Green Deployments

Zero-downtime deployments via ECS blue-green: deploy new task definition, run health checks, shift traffic, terminate old tasks. Currently a simple rolling update.

Distributed Tracing (OpenTelemetry)

Add OpenTelemetry instrumentation to trace a single request across: Axios → ALB → FastAPI → PostgreSQL → SQS → Worker. Send traces to Jaeger or AWS X-Ray for latency debugging.

Search (Elasticsearch)

Full-text search on incident descriptions and evidence content. PostgreSQL full-text search works for small datasets but Elasticsearch handles millions of documents with better relevance scoring.

Alert Fatigue Reduction

Implement ML-based alert correlation to group related incidents into a single case. Reduces analyst cognitive load when a single attack campaign generates hundreds of individual alerts.

Mobile App

React Native mobile app for on-call analysts — push notifications for critical incidents, incident status updates, quick triage actions.

Section Summary

LBRO's next evolution would prioritise real-time WebSocket updates, advanced ML models, multi-tenancy, enterprise SSO, E2E testing, and distributed tracing. Each improvement addresses a specific limitation of the current architecture and follows industry best practices.

Interview Questions

- How would you add real-time updates to LBRO?
- What is the challenge of multi-tenancy in PostgreSQL?
- What is OpenTelemetry?
- How would you handle 10 million incidents in PostgreSQL?

Common Mistakes

[!] Treating improvements as 'nice-to-have' — performance and scalability issues compound over time.
[!] Adding multi-tenancy as an afterthought — retrofitting tenant isolation into an existing schema is complex.
[!] Skipping E2E tests because unit tests exist — unit tests don't catch integration failures.

Things to Remember

[v] WebSockets for real-time updates beat polling
[v] Multi-tenancy: organisation_id FK + PostgreSQL RLS
[v] OpenTelemetry: distributed tracing across all services
[v] Elasticsearch for full-text search at scale